

RPS-GTok: A Graph Tokenizer for Reconstructable and Permutation-Stable Graph Tokens

Anonymous Author(s)

Abstract

Tokenizers make Transformer-based models applicable across modalities by converting raw inputs into discrete sequences. Graph tokenization remains challenging because graphs have no inherent linear order and may contain heterogeneous structure. The explicit reconstruction path and permutation-stable token emission are key requirements for graph tokens, yet existing methods have difficulty satisfying both by construction. We introduce Reconstructable and Permutation-Stable Graph Tokenizer (RPS-GTok) to generate graph-token instances that respect graph topology. RPS-GTok compiles each graph into a reconstructable primitive-operation sequence and then emits graph tokens whose expansions preserve that sequence. Permutation stability is enforced by graph canonical relabeling of RPS-GTok, which removes dependence on incidental node identifiers before token emission. Reconstructability is supported by complete graph-token instances of RPS-GTok that retain the arguments needed for expansion, while the input to the Transformer is the ordered identifier sequence derived from those instances. Across 21 graph datasets, RPS-GTok averages 0.298 Tok./edge and 1.522 Tok./node. On 2,602 sampled attributed graphs, RPS-GTok achieves the best overall trade-off among the compared methods, combining low token cost with 100% reconstructability and permutation stability. On downstream topological tasks, RPS-GTok achieves better performance on both classification and regression tasks with the respective Transformer-based model settings, compared with competing tokenization methods. The code of RPS-GTok is released at https://anonymous.4open.science/r/RPS_GTok.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Mathematics of computing** → *Graph theory*.

Keywords

Transformers, Graph Tokenizer, Graph topology, Reconstructability, Permutation stability

ACM Reference Format:

Anonymous Author(s). 2027. RPS-GTok: A Graph Tokenizer for Reconstructable and Permutation-Stable Graph Tokens. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '27)*, August 2027, Location withheld for anonymous draft. ACM, New York, NY, USA, 14 pages.

1 Introduction

Transformer-based models usually take discrete token sequences converted from raw data as input [3, 13, 27, 34, 36, 44, 50]. For text-based Transformers, text tokenizer methods such as Byte Pair Encoding, SentencePiece, and WordPiece segment text into subword tokens while preserving their natural linear order [10, 15, 29, 39, 41].

KDD '27, Location withheld for anonymous draft
2027.

Image tokenizers typically partition images into fixed-size patches and convert them into patch-level tokens, while retaining the natural 2D grid order through positional encodings [11]. These tokenizers are effective because the inputs already provide a stable sequence or grid structure. Text is inherently sequential data with a canonical one-dimensional order. Images are regular 2D grid-structured data, where patches follow a fixed spatial arrangement.

Graphs do not come with a natural left-to-right order like text or fixed spatial positions like image patches. They may contain undirected edges, multiedges, self-loops, isolated nodes, cycles, and hubs. For N unordered nodes and M parallel edges, there can be $N!$ node orders and $M!$ edge orders. Self-loops and isolated nodes can be placed freely, and traversal-based orderings of cycles and hubs remain sensitive to the chosen start node.

Once a tokenizer emits a token sequence, both the tokens and their order become part of what a Transformer-based model observes. To ensure the Transformer-based model can learn graph topological structure from graph tokens, a graph tokenizer needs to satisfy reconstructability and permutation stability. Reconstructability means that the emitted graph tokens can be expanded back to a graph topologically equivalent to the source graph under the prescribed paradigm; permutation stability means that incidental node relabeling leaves the emitted token sequence unchanged. These properties determine not only whether the resulting sequence preserves graph structure for downstream models, but also whether graph tokens can be generated from graphs without depending on incidental node identifiers.

We introduce the Reconstructable and Permutation-Stable Graph Tokenizer (RPS-GTok) to address these challenges. RPS-GTok consists of four core components: a graph canonical relabeling component assigns canonical identifiers from graph structure to remove dependence on original node identifiers; a reconstructable primitive compilation component turns the canonical graph into a reconstructable sequence of primitive operations; a vocabulary-guided primitive fragment formation component identifies recurring primitive fragments and matches them to frozen vocabulary entries; and a graph-token formation component encodes the matched primitive fragments as graph tokens whose expansions preserve the underlying primitive operations.

In this work, our contributions can be summarized as follows:

- **Primitive-Operation-Based Graph Token.** We introduce a graph-token formulation based on primitive operations. It forms graph tokens by merging reusable primitive operation fragments and preserving uncovered graph information through primitive operations.
- **Graph Tokenization Algorithm of RPS-GTok.** We design a graph tokenization algorithm around reconstructability and permutation stability, producing RPS-GTok graph-token sequences that preserve the source graph topology under the prescribed graph schema.

- **Performance Gains On Downstream Topological Tasks.** The RPS-GTok improves Transformer-based model performance on both topological classification and regression tasks compared with other methods.

2 Related Work

2.1 Graph Tokenization Methods

Existing graph tokenization methods can be roughly organized by the graph-to-token pathway they follow: serialization-based methods map graphs to ordered symbol sequences, structure-unit-based methods map graphs to explicit graph units, vector-quantization-based methods map graph representations to discrete codebook tokens, and pooling-based methods map graph representations to continuous graph-level tokens.

Serialization-based methods [14] convert a graph into an ordered symbolic sequence, such as node records, edge records, or triples. Related graph-text systems [51] and graph chain-of-thought methods [22] follow the same route. Edge lists, adjacency lists, and JSON records are common textual encodings, while GraphML [2] and DOT [16] provide explicit graph languages. RDF triples [7], SMILES [47], and SELFIES [26] follow a similar serialization route. Recent methods such as Graph Generative Pre-trained Transformer (G2PT) [5] and Graph Tokenization for Bridging Graphs and Transformers [18] further adapt graph serialization for Transformer-based models. These graph tokens can be discrete and reversible, but their sequence often depends on identifiers, traversal choices, or serialization syntax.

Structure-unit-based tokenization methods [49] construct graph tokens from explicit graph units, such as k-hop neighborhoods, random-walk-sampled subgraphs, or frequent subgraph patterns. Frequent subgraph mining [30] and Weisfeiler-Lehman-style refinements [42] are representative ways to obtain such units. TokenGT [23], NAGphormer [4], HIGHT [6], and DemoDiff with Node Pair Encoding [32] are representative examples. Such tokenization can produce structurally meaningful graph tokens. However, because the units are selected rather than exhaustively constructed, parts of the source graph may remain uncovered.

Vector-quantization-based methods discretize GNN-encoded node representations into codebook indices. A representative example is Graph Quantized Tokenizer (GQT) [46], which uses Residual Vector Quantization (RVQ) to produce hierarchical discrete tokens. Pooling-based methods [24], by contrast, aggregate node representations into a fixed number of continuous graph-level tokens, such as graph pooling tokens. However, these representations are either quantized latent codes or continuous pooled representations, rather than reconstructable graph tokens.

RPS-GTok differs from these methods by attaching explicit expansion arguments to each emitted token instance, so the token remains tied to a recoverable graph fragment rather than a latent code or pooled summary.

2.2 Graph Isomorphism

Graph isomorphism exposes a basic challenge for graph tokenization. Two graphs may represent the same topology while differing only in incidental node identifiers; even for a fixed graph, different traversal choices may produce different serialized orders. A

Transformer-based model should learn the same topological features from such equivalent graph structures, rather than learning inconsistent representations from different node orders or traversal sequences. If equivalent graphs are converted into inconsistent token sequences, the Transformer-based model may treat equivalent topologies as different patterns and learn serialization artifacts rather than stable graph structure.

Existing work usually handles permutation effects at the model level. Graph neural networks achieve permutation equivariance through message passing [38] and widely used architectures such as GCN [25], GraphSAGE [19], and GAT [45]. Related aggregation mechanisms appear in GIN [48], graph convolutional models [8, 12], neural message passing [17], and relational reasoning [1]. These models can make representations or predictions stable when an already constructed graph input is permuted. However, this model-level treatment does not transfer directly to Transformer-based models, such as LLM-style architectures [44] and BERT-style encoders [10], because these models consume the emitted token sequence as ordered input. If the tokenizer assigns different sequences to isomorphic graphs, Transformer-based models may treat these sequence differences as meaningful structural variation. Therefore, the graph token sequence should satisfy input-level permutation stability before it is processed by a Transformer-based model.

RPS-GTok differs by making this stability a tokenizer-level property: the sequence presented to the Transformer-based model is already independent of incidental node identifiers.

3 Primitive-Operation-Based Graph Token

3.1 Prescribed Graph Paradigm

The prescribed graph paradigm S is the contract that defines what graph information RPS-GTok retains. Reconstructability is assessed relative to S . Let $\mathcal{G}$ denote the set of attributed graphs that satisfy S . RPS-GTok represents an input attributed graph G in the form

$$G = (g, d_G, t_G, \mathcal{V}, \mathcal{E}, A_G), \quad (1)$$

where g is the graph identifier, d_G is the data-source category, t_G is the graph type, $\mathcal{V}$ is the node tuple set, $\mathcal{E}$ is the edge tuple multiset, and A_G stores graph attributes. Node tuples v and edge tuples e are

$$v = (\iota_v, \tau_v, A_v, \xi_v), \quad e = (s_e, t_e, \tau_e, \delta_e, k_e, A_e). \quad (2)$$

For node tuple v , ι_v is a node identifier, τ_v is a node type, A_v stores node attributes, and ξ_v is a stable identifier. For edge tuple e , s_e, t_e are endpoints, τ_e is an edge type, δ_e is the direction flag, k_e is a stable multiedge key derived from graph content rather than input edge insertion order, and A_e stores edge attributes. Under S , each node identifier must be unique, and each edge endpoint must refer to an existing node. The paradigm supports directed and undirected edges, multiedges, self-loops, and isolated nodes.

The reconstruction contract is limited to graph information retained by S . The fields g and d_G are sequence-level metadata used for provenance, while t_G and A_G are retained graph-level fields when S keeps them. The stable identifier ξ_v is an auxiliary per-node reference used for canonicalization and auditing; unless S explicitly retains node identity, it is not treated as a reconstruction target.

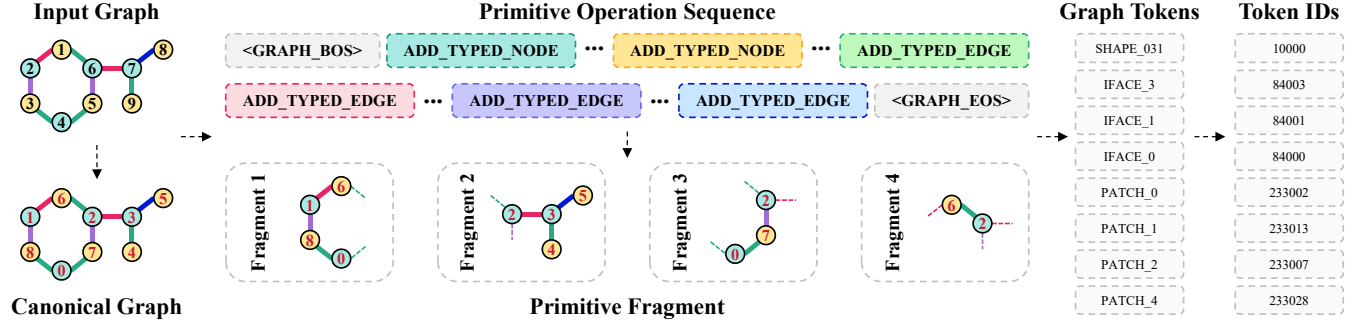


Figure 1: Overview of the RPS-GTok graph token generation process.

3.2 Graph Editing and Primitive Operation

Graph editing represents a graph that satisfies S as an ordered sequence of primitive operations. Applying this sequence to an empty graph state reconstructs a graph equivalent to the source graph under S . Each primitive operation is a low-level executable command that records one retained element or relation, such as a typed node, an edge, a structural flag, or an attribute.

A primitive operation p has an operation field op and operation-specific arguments $args$:

$$p = (op, args), \quad p \in \mathcal{P}_{\text{prim}}. \quad (3)$$

The operation field set $\mathcal{O}_{\text{prim}}$ for primitive operations consists of $\mathcal{O}_{\text{node}}$ and $\mathcal{O}_{\text{edge}}$, where $\mathcal{P}_{\text{prim}} = \{(op, args) : op \in \mathcal{O}_{\text{prim}}\}$ is the primitive operation space:

$$\mathcal{O}_{\text{node}} = \{\text{ADD_TYPED_NODE}, \text{ISOLATED_NODE}, \text{SET_NODE_ATTR}\}. \quad (4)$$

$$\mathcal{O}_{\text{edge}} = \{\text{ADD_TYPED_EDGE}, \text{SELF_LOOP}, \text{DIRECTED_ATTACH}, \text{MULTI_EDGE}, \text{SET_EDGE_ATTR}\}. \quad (5)$$

$$\mathcal{O}_{\text{prim}} = \mathcal{O}_{\text{node}} \cup \mathcal{O}_{\text{edge}}. \quad (6)$$

In $\mathcal{O}_{\text{node}}$, the `ADD_TYPED_NODE` creates typed nodes, the `ISOLATED_NODE` marks isolated nodes, and the `SET_NODE_ATTR` attaches retained node attributes. In $\mathcal{O}_{\text{edge}}$, the `ADD_TYPED_EDGE` creates typed edges, the `SELF_LOOP` records self-loops, the `DIRECTED_ATTACH` records directed attachments, the `MULTI_EDGE` records multiedges, and the `SET_EDGE_ATTR` attaches retained edge attributes.

Boundary markers $\mathcal{P}_{\text{bd}}$ delimit a primitive operation sequence. The primitive operations between them carry the complete descriptive information of a graph:

$$\mathcal{P}_{\text{bd}} = \{(< \text{GRAPH_BOS} >, args_{\text{bos}}), (< \text{GRAPH_EOS} >, \emptyset)\}. \quad (7)$$

Here $args_{\text{bos}}$ stores sequence-level metadata, including the graph identifier, data-source category, graph type, and any retained graph-level attributes required by S . Retained graph-level fields are decoded from the boundary marker, whereas non-boundary primitive operations reconstruct node and edge structure. Graph editing enforces local preconditions on each operation, and invalid primitive programs are rejected rather than partially executed.

3.3 Graph Vocabulary

The graph vocabulary $\mathcal{A}_{\theta}$ is defined under S and built offline from training graphs $\mathcal{D}_{\text{train}} \subseteq \mathcal{G}$. It collects recurring structural patterns over the graph information retained by S , including retained node tuples, edge tuples, attributes, local substructures, graph-level shape summaries, boundary signatures, and primitive fragments.

The graph vocabulary $\mathcal{A}_{\theta}$ contains entry families with different roles. Special entries $\mathcal{A}_{\text{spec}}$ reserve sequence-boundary identifiers. Primitive entries $\mathcal{A}_{\text{prim}}$ index operation fields in $\mathcal{O}_{\text{prim}}$. Substructure entries $\mathcal{A}_{\text{sub}}$ store substructure signatures defined under S , and macro entries $\mathcal{A}_{\text{macro}}$ store frequent primitive operation fragments. Shape entries $\mathcal{A}_{\text{shape}}$ store graph-level shape signatures. Interface entries $\mathcal{A}_{\text{interface}}$ store boundary signatures of retained substructures. Attribute entries $\mathcal{A}_{\text{attr}}$ and domain entries $\mathcal{A}_{\text{domain}}$ store retained metadata specified by S .

$$\mathcal{A}_{\theta} = \mathcal{A}_{\text{spec}} \cup \mathcal{A}_{\text{prim}} \cup \mathcal{A}_{\text{sub}} \cup \mathcal{A}_{\text{macro}} \cup \mathcal{A}_{\text{shape}} \cup \mathcal{A}_{\text{interface}} \cup \mathcal{A}_{\text{attr}} \cup \mathcal{A}_{\text{domain}}. \quad (8)$$

To build $\mathcal{A}_{\text{sub}}$, vocabulary construction counts substructure signatures in the training split $\mathcal{D}_{\text{train}}$. For each G_i , let $\mathcal{S}_{G_i}$ be the candidate substructures. Each candidate substructure $U \in \mathcal{S}_{G_i}$ contributes an aggregate signature $\chi(U)$, composed of the attribute, typed-structure, and topology signatures used later for vocabulary matching. Signatures that recur frequently across the training split are retained as substructure entries.

$$\bar{\epsilon}_{\theta} : \mathcal{A}_{\theta} \times \mathcal{B}_{\theta} \rightarrow (\mathcal{P}_{\text{bd}} \cup \mathcal{P}_{\text{prim}})^*. \quad (9)$$

Here $\mathcal{B}_{\theta}$ is the token-argument payload space, and $\bar{\epsilon}_{\theta}$ expands each entry-payload pair into boundary markers or primitive operations. The payload stores only the minimal token-local arguments required by $\bar{\epsilon}_{\theta}$; it does not duplicate the expanded primitive operation sequence itself. Frequent primitive operation fragments are retained as macro entries in $\mathcal{A}_{\text{macro}}$. Once constructed, $\mathcal{A}_{\theta}$ and $\bar{\epsilon}_{\theta}$ are frozen before inference.

4 Graph Tokenization Algorithm of RPS-GTok

When RPS-GTok is applied to a attributed graph under S , the emitted graph tokens satisfy:

- **Permutation-Stable.** Graph tokens do not depend on the original order of node identifiers.
- **Reconstructable.** Graph tokens have the primitive-operation expansion, and graph information not covered by the graph vocabulary can still be represented using primitive operations.

Algorithm 1: RPS-GTok Tokenization Workflow

Input: Attributed graph G , prescribed graph paradigm S , graph vocabulary $\mathcal{A}_\theta$, expansion map $\hat{e}_\theta$

Output: Emitted graph-token sequence $Z = (z_1, \dots, z_L)$

- 1 $G^c \leftarrow \kappa_S(G)$
- 2 $PC \leftarrow C_S(G^c)$
- 3 Initialize $\mathcal{U}_{\text{ret}} \leftarrow \emptyset$, $E_{\text{emit}} \leftarrow \emptyset$, and Z with the $\langle \text{GRAPH_BOS} \rangle$ token
- 4 Form candidate node tuple sets $U \subseteq \mathcal{V}^c$
- 5 Include edge-level and isolated-node candidates to preserve graph coverage
- 6 Sort candidates by canonical keys and deterministic tie-breaking rules
- 7 **foreach** candidate node tuple set U in sorted order **do**
- 8 Compute $E_{\text{in}}(U)$, $E_\partial(U)$, and $P(U)$ by Eq. (21)
- 9 **if** U does not satisfy Eq. (22) **then**
- 10 **continue**
- 11 Compute the primitive fragment $PC_U \leftarrow \text{Subseq}_S(PC; U, E_{\text{in}}(U))$
- 12 Query $\mathcal{A}_\theta$ for matching substructure and macro entries using PC_U , $\sigma_{\text{attr}}(U)$, $\sigma_{\text{typed}}(U)$, and $\sigma_{\text{topo}}(U)$ in deterministic order
- 13 **if** a vocabulary entry $a_U \in \mathcal{A}_\theta$ is matched **then**
- 14 link a_U to PC_U
- 15 Compute C_E by Eq. (24) and $\Delta(U \mid C_E, \mathcal{U}_{\text{ret}})$ by Eq. (25)
- 16 **if** $\Delta(U \mid C_E, \mathcal{U}_{\text{ret}}) > 0$ **then**
- 17 add U to $\mathcal{U}_{\text{ret}}$
- 18 Sort $\mathcal{U}_{\text{ret}}$ by canonical fragment order and deterministic tie-breaking rules
- 19 **foreach** $U \in \mathcal{U}_{\text{ret}}$ in sorted order **do**
- 20 Compute $PC_U^+ \leftarrow \text{Unemitted}(PC_U; N_{\text{emit}}, E_{\text{emit}})$
- 21 **if** PC_U is linked to a_U **then**
- 22 Construct the minimal token-local arguments a_U from $P(U)$, retained attributes, and the expansion parameters required for PC_U^+
- 23 emit $z_U = (a_U, a_U)$ with $\exp_\theta(z_U) = PC_U^+$
- 24 **else**
- 25 emit primitive-operation entries for PC_U^+
- 26 Update N_{emit} and E_{emit}
- 27 Emit primitive-operation entries for still-unemitted graph content
- 28 Append the $\langle \text{GRAPH_EOS} \rangle$ token to Z
- 29 Audit $I_S(\text{Exp}_\theta(Z)) \equiv_S G^c$
- 30 **return** Z

Figure 1 illustrates the graph token generation process of RPS-GTok, and Algorithm 1 presents the graph tokenization algorithm of RPS-GTok.

4.1 Graph Canonical Relabeling

Given an attributed graph $G \in \mathcal{G}$ under S , graph canonical relabeling produces a canonical graph G^c by assigning node identifiers according to graph information rather than incidental input identifiers. RPS-GTok implements this step as an attributed canonical labeling procedure [33]. The procedure first builds deterministic node and edge-pair signatures, then refines node ordering keys, and finally resolves remaining ties by selecting the lexicographically minimal graph serialization.

Deterministic attribute serialization converts every retained attribute set into a schema-fixed representation before token generation. For an attribute set A , $R(A)$ denotes the ordered key-value string produced by sorting retained keys, normalizing missing values, and applying the value formatting specified by S ; $Q(A)$ denotes the corresponding quantized representation when S requires attribute quantization. Thus, attributes with the same retained content receive the same serialized form independent of input row order.

For a node tuple v , the node-content signature b_v is

$$b_v = (\tau_v, Q(A_v), R(A_v)). \quad (10)$$

Here τ_v is the node type, $Q(A_v)$ is the quantized node-attribute representation, and $R(A_v)$ is the deterministic raw-attribute string

retained for v . The first ordering key of v is therefore initialized by the graph information attached to the node itself.

For a node pair (u, v) , let $\mathcal{E}_{uv}$ be the edge tuples between u and v under S . The pair signature ρ_{uv} is

$$\rho_{uv} = \text{sort}\{(\eta, \tau_e, \delta_e, k_e, Q(A_e), R(A_e)) : e \in \mathcal{E}_{uv}\}. \quad (11)$$

Here η records the endpoint role, τ_e is the edge type, δ_e is the direction flag, k_e is a stable multiedge key, $Q(A_e)$ is the quantized edge-attribute representation, and $R(A_e)$ is the deterministic raw edge-attribute serialization retained under S . The key k_e is retained when the graph provides stable parallel-edge identity; otherwise it is assigned as a deterministic rank after sorting parallel-edge records by $(\eta, \tau_e, \delta_e, Q(A_e), R(A_e))$. The sorting step gives a deterministic order to multiple edge descriptors between the same node pair, so multiedges and retained edge attributes enter canonical relabeling without depending on input edge order.

Starting from $q_v^{(0)} = b_v$, RPS-GTok iteratively updates the ordering key of each node tuple by combining its current key with the sorted keys and pair signatures of other node tuples:

$$q_v^{(t+1)} = \text{KeyUpdate}\left(q_v^{(t)}, \text{sort}\{(q_u^{(t)}, \rho_{uv}) : u \in \mathcal{V}\}\right). \quad (12)$$

Here $\mathcal{V}$ is the node tuple set of G , $q_v^{(t)}$ is the ordering key of v after the t -th update, and KeyUpdate is a deterministic refinement function. This refinement follows the principle of color-refinement-based canonical labeling: nodes with different local attributed structure receive different ordering keys, while structurally indistinguishable nodes remain tied. The update stops when the ordering keys become stable. Let $\mathcal{Q}^* = \{q_v^* : v \in \mathcal{V}\}$ denote the stable ordering-key set.

If multiple node tuples share the same stable ordering key, only their within-key order remains unresolved. RPS-GTok resolves this ambiguity through an individualization-refinement search. It branches only inside tied groups, temporarily places one tied node into a singleton position, refines the ordering keys again, and compares the resulting candidate orders. For a candidate node order $\omega = (v_1, \dots, v_n) \in \Omega(\mathcal{Q}^*)$, where $\Omega(\mathcal{Q}^*)$ is the set of node orders consistent with the stable ordering-key set, the rank map $r_\omega(v_i) = i$ assigns node tuple v_i its candidate canonical rank.

Each candidate order is compared by a deterministic serialization under S :

$$\text{ser}_S(\omega(G)) = \left([(i, \tau_{v_i}, Q(A_{v_i}), R(A_{v_i}))]_{i=1}^n, \text{sort}\{(r_\omega(s_e), r_\omega(t_e), \tau_e, \delta_e, k_e, Q(A_e), R(A_e)) : e \in \mathcal{E}\} \right). \quad (13)$$

In Eq. (13), ser_S writes node tuples according to their candidate ranks. For an edge tuple e , the original endpoint identifiers s_e and t_e are replaced by the candidate ranks $r_\omega(s_e)$ and $r_\omega(t_e)$, while $Q(A_e)$ and $R(A_e)$ keep both quantized and raw retained edge-attribute content visible to tie-breaking. Different candidate orders are therefore compared using canonical ranks and retained graph content rather than raw node identifiers.

The final node order is the candidate order with the lexicographically minimal serialization:

$$\omega_G = \arg \min_{\omega \in \Omega(\mathcal{Q}^*)} \text{ser}_S(\omega(G)). \quad (14)$$

The selected order ω_G defines the canonical graph

$$G^c = \kappa_S(G) = (g, d_G, t_G, \mathcal{V}^c, \mathcal{E}^c, A_G). \quad (15)$$

Here $\mathcal{V}^c = \omega_G(\mathcal{V})$ is the node tuple set after canonical ordering, and $\mathcal{E}^c = \omega_G(\mathcal{E})$ is the edge tuple multiset after endpoint relabeling. Canonical identifiers n_i are assigned according to ω_G , undirected endpoints are normalized, and parallel edges are sorted by retained keys and attributes. For any incidental relabeling π of node identifiers, graph canonical relabeling satisfies

$$\kappa_S(\pi(G)) = \kappa_S(G). \quad (16)$$

Together with deterministic candidate selection, tie-breaking, and emission, canonical relabeling yields permutation-stable tokenization.

4.2 Reconstructable Primitive Compilation

After graph canonical relabeling fixes G^c , reconstructable primitive compilation deterministically converts the canonical graph into the ordered primitive operation sequence used by Graph editing. Let C_S denote the compilation procedure that converts G^c into a primitive operation sequence under S . The resulting sequence is

$$PC = C_S(G^c) = [p_{\text{bos}}, p_1, \dots, p_T, p_{\text{eos}}], \quad (17)$$

$$p_i \in \mathcal{P}_{\text{prim}}, \quad i = 1, \dots, T.$$

Here PC is the primitive operation sequence, and p_{bos} and p_{eos} are the boundary markers in $\mathcal{P}_{\text{bd}}$. The index i ranges over the non-boundary primitive operations, and each $p_i = (op_i, args_i)$ belongs to $\mathcal{P}_{\text{prim}}$. Compilation follows the canonical tuple order fixed in G^c . For canonical node tuples, it emits node operations from $\mathcal{O}_{\text{node}}$ for node creation, isolated-node markers, and retained node attributes. For canonical edge tuples, it emits edge operations from $\mathcal{O}_{\text{edge}}$ for structural markers, edge creation, and retained edge attributes. Thus, the order of PC is determined by G^c and S .

Reconstructability is checked by the Graph editing state induced by PC , not by the surface form of the input file. Let Γ_0 be the graph state initialized from p_{bos} , and let Γ_i be the state after applying the i -th non-boundary primitive operation:

$$\Gamma_0 = \text{Init}_S(p_{\text{bos}}), \quad \Gamma_i = \text{Apply}_S(\Gamma_{i-1}, p_i), \quad i = 1, \dots, T. \quad (18)$$

Here Init_S decodes the sequence metadata and any retained graph-level fields carried by p_{bos} , and Apply_S is the Graph editing update for one primitive operation under the prescribed graph paradigm S . For PC , $\mathcal{I}_S(PC)$ denotes the state produced after the closing marker is checked:

$$\mathcal{I}_S(PC) = \text{Close}_S(\Gamma_T, p_{\text{eos}}). \quad (19)$$

The compilation is reconstructable when $\mathcal{I}_S(PC)$ contains the same retained graph information as G^c . Let R_G^S , R_V^S , R_E^S , and R_A^S extract the retained graph-level fields, canonical node rows, edge rows, and retained attribute rows specified by S . Reconstructable primitive compilation requires

$$\begin{aligned} R_G^S(\mathcal{I}_S(PC)) &= R_G^S(G^c), & R_V^S(\mathcal{I}_S(PC)) &= R_V^S(G^c), \\ R_E^S(\mathcal{I}_S(PC)) &= R_E^S(G^c), & R_A^S(\mathcal{I}_S(PC)) &= R_A^S(G^c). \end{aligned} \quad (20)$$

Therefore, every node, edge, structural flag, and retained attribute in G^c must be represented by boundary metadata or a primitive operation in PC ; otherwise, the compilation is not reconstructable.

4.3 Vocabulary-Guided Primitive Fragment Formation

Vocabulary-guided primitive fragment formation takes the canonical graph G^c , the primitive operation sequence $PC = C_S(G^c)$, and the frozen graph vocabulary $\mathcal{A}_\theta$ as inputs. It identifies canonical substructures that can be represented by ordered primitive operation fragments. Because the inputs are G^c and PC , candidate formation is independent of raw node identifiers.

For a candidate node tuple set $U \subseteq \mathcal{V}^c$, let $\iota(U) = \{t_v : v \in U\}$ be the canonical node identifiers in U . The internal edges, boundary edges, and ports of U are

$$\begin{aligned} E_{\text{in}}(U) &= \{e \in \mathcal{E}^c : s_e, t_e \in \iota(U)\}, \\ E_\partial(U) &= \{e \in \mathcal{E}^c : |\{s_e, t_e\} \cap \iota(U)| = 1\}, \\ P(U) &= \text{Ports}_S(U, E_\partial(U)). \end{aligned} \quad (21)$$

Here $E_{\text{in}}(U)$ contains edge tuples whose endpoints both lie in $\iota(U)$, $E_\partial(U)$ contains edge tuples with exactly one endpoint in $\iota(U)$, and $P(U)$ records the boundary ports induced by $E_\partial(U)$ under the prescribed graph paradigm S . Candidate regions are proposed from local neighborhoods, dense cores, bridge regions, type-aware neighborhoods, edge-level regions, and isolated nodes. Each candidate is subject to vocabulary-guided limits:

$$|U| \leq \min(m, r), \quad |P(U)| \leq b_{\ell(U)}. \quad (22)$$

In Eq. (22), m is the maximum fragment size, r is the maximum number of canonical graph nodes allowed in one candidate, $\ell(U)$ is the candidate source category, and $b_{\ell(U)}$ is the port limit for that category. Edge-level and isolated-node candidates remain admissible, so graph portions not covered by larger substructures can still be represented by primitive fragments. This design keeps the emitted graph tokens reconstructable.

Each candidate node tuple set U induces a primitive fragment PC_U . The fragment is the order-preserving subsequence of PC whose primitive operations describe the node tuples in U , the internal edges in $E_{\text{in}}(U)$, and their retained attributes:

$$PC_U = \text{Subseq}_S(PC; U, E_{\text{in}}(U)). \quad (23)$$

In Eq. (23), Subseq_S preserves the order inherited from PC and selects primitive operations according to the graph information retained by S .

Vocabulary-guided matching then checks whether candidate U has a recurring structural pattern in $\mathcal{A}_\theta$. For each U , RPS-GTok computes $\sigma_{\text{attr}}(U)$, $\sigma_{\text{typed}}(U)$, and $\sigma_{\text{topo}}(U)$, and queries substructure entries in that order. A successful query links the matched vocabulary entry $a_U \in \mathcal{A}_\theta$ to the induced primitive fragment PC_U .

Candidate retention decides which candidates are kept. The goal is to keep candidates that add useful primitive fragments, avoid repeatedly selecting candidates that describe the same edge tuples, and preserve graph coverage. Let $\mathcal{U}_{\text{ret}}$ be the candidate node tuple sets that have already been retained. The covered-edge set is

$$C_E = \bigcup_{U' \in \mathcal{U}_{\text{ret}}} E_{\text{in}}(U'). \quad (24)$$

In Eq. (24), U' ranges over retained candidate node tuple sets in $\mathcal{U}_{\text{ret}}$, $E_{\text{in}}(U')$ is the internal edge tuple set induced by U' , and C_E

is the union of internal edge tuples already covered by retained candidates. For a new candidate U , the marginal utility is

$$\Delta(U \mid C_E, \mathcal{U}_{\text{ret}}) = \lambda_E |E_{\text{in}}(U) \setminus C_E| + s_{\text{loc}}(U) + \lambda_H \mathbf{1}\{U \text{ matches } \mathcal{A}_\theta\} - \lambda_J J(U, \mathcal{U}_{\text{ret}}). \quad (25)$$

Here $\lambda_E, \lambda_H, \lambda_J$ are fixed weights. The first term rewards edge tuples in $E_{\text{in}}(U)$ that are not yet covered by C_E . The local score $s_{\text{loc}}(U)$ uses the candidate information already defined for U , including density, type consistency, attribute consistency, source category $\ell(U)$, boundary-port cost from $P(U)$, and out-of-distribution risk. The indicator $\mathbf{1}\{U \text{ matches } \mathcal{A}_\theta\}$ rewards candidates that match the graph vocabulary, and $J(U, \mathcal{U}_{\text{ret}})$ penalizes overlap with retained candidates. Candidates with positive marginal utility are added to $\mathcal{U}_{\text{ret}}$. Edge-level and isolated-node candidates remain admissible, so graph parts not covered by larger substructures are still represented by primitive fragments.

4.4 Graph Tokens Formed from Primitive Fragments

Given a retained primitive fragment and its matched vocabulary entry, RPS-GTok forms a graph-token instance by using the vocabulary entry as the token identifier and storing the arguments needed to reconstruct the primitive-operation fragment. An emitted graph token is

$$z = (x, \alpha), \quad x \in \mathcal{A}_\theta, \quad \alpha \in \mathcal{B}_\theta. \quad (26)$$

Here z is a graph token, x is the vocabulary identifier consumed by the model, and α stores token-specific arguments such as ports, attributes, and other minimal instance data needed for expansion. The expansion of z is

$$\text{exp}_\theta(z) = \bar{e}_\theta(x; \alpha) \in (\mathcal{P}_{\text{bd}} \cup \mathcal{P}_{\text{prim}})^*. \quad (27)$$

RPS-GTok avoids duplicate construction by maintaining emitted node and edge sets N_{emit} and E_{emit} . For a retained candidate node tuple set U , the emitted part of its primitive fragment is

$$PC_U^+ = \text{Unemitted}(PC_U; N_{\text{emit}}, E_{\text{emit}}). \quad (28)$$

Here N_{emit} and E_{emit} record canonical nodes and edge tuples already emitted. The operator Unemitted selects the order-preserving part of PC_U whose graph content has not been emitted.

If PC_U is linked to a vocabulary entry $a_U \in \mathcal{A}_\theta$, RPS-GTok emits a graph token $z_U = (a_U, \alpha_U)$ whose expansion map reconstructs PC_U^+ as $\text{exp}_\theta(z_U) = PC_U^+$. If no learned vocabulary entry represents the required graph information, RPS-GTok emits primitive-operation entries whose concatenated expansions equal PC_U^+ . Thus, graph information not matched by the graph vocabulary remains represented by primitive operations.

The emitted sequence is $Z = (z_1, \dots, z_L)$, where L is the number of graph tokens, and its identifier projection consumed by the Transformer is $X = (x_1, \dots, x_L)$. The expanded primitive operation sequence of Z is

$$\text{Exp}_\theta(Z) = \text{exp}_\theta(z_1) \parallel \dots \parallel \text{exp}_\theta(z_L). \quad (29)$$

Here $\parallel$ denotes sequence concatenation. The sequence is reconstructable when its expansion is a boundary-delimited primitive operation sequence and Graph editing reconstructs the canonical graph under S :

$$\text{Exp}_\theta(Z) \in (\mathcal{P}_{\text{bd}} \cup \mathcal{P}_{\text{prim}})^*, \quad I_S(\text{Exp}_\theta(Z)) \equiv_S G^c. \quad (30)$$

Here $I_S(\cdot)$ denotes the final graph state produced by applying Graph editing to a primitive operation sequence under S . The notation $\equiv_S$ denotes the equalities in Eq. (20).

Since $G^c = \kappa_S(G)$, the same condition establishes reconstructability for the source graph under the prescribed graph paradigm S . Figure 2 shows this reconstruction path from tokens back to the canonical graph and the source graph.

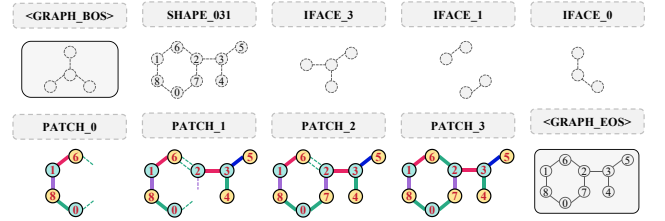


Figure 2: Reconstruction path from graph tokens.

4.5 Formal Properties of RPS-GTok

Let $T_{S, \mathcal{A}_\theta}$ be the complete encoder, $\Pi_S(G)$ the S -preserving incidental relabelings of G , and $\perp$ an encoding failure.

$$I_S(C_S(\kappa_S(G))) \equiv_S \kappa_S(G), \quad G \in \mathcal{G}. \quad (31)$$

Equation (31) states primitive completeness: the compiled primitive program contains all retained canonical graph information and respects the dependencies required by Graph editing.

$$\text{Exp}_\theta(T_{S, \mathcal{A}_\theta}(G)) = C_S(\kappa_S(G)), \quad G \in \mathcal{G}. \quad (32)$$

Equation (32) states merge preservation: fragment tokens compact only subsequences whose expansion matches the underlying primitive program.

$$T_{S, \mathcal{A}_\theta}(G) = Z = (z_1, \dots, z_L) \neq \perp, \quad L < \infty, \quad (33)$$

$$\text{Exp}_\theta(Z) \in (\mathcal{P}_{\text{bd}} \cup \mathcal{P}_{\text{prim}})^*, \quad G \in \mathcal{G}.$$

Equation (33) states total encodability: every graph admitted by S is mapped to a finite graph-token sequence whose expansion remains a valid boundary-delimited primitive-operation sequence, so limited vocabulary coverage can lengthen the sequence but cannot make retained graph content unencodable.

$$T_{S, \mathcal{A}_\theta}(\pi(G)) = T_{S, \mathcal{A}_\theta}(G), \quad \pi \in \Pi_S(G). \quad (34)$$

Equation (34) states permutation stability: canonical relabeling and deterministic emission remove dependence on incidental node identifiers from the emitted graph-token sequence.

$$\text{cost}_{\text{prim}}(G) = O(n + m_e + n_a),$$

$$\text{cost}_{\text{frag}}(G) = O(c(G) \log c(G)). \quad (35)$$

Equation (35) gives the explicit computational boundary for $n = |V|$, $m_e = |E|$, retained attributes n_a , and candidates $c(G)$; canonical relabeling may still branch on highly symmetric graphs.

Table 1: Graph datasets and graph-size statistics.

Dataset	#G	Avg. $ V $	Avg. $ E $	Max. $ V $	Max. $ E $
AST-CFG-CPG	3,761	1023.1	1022.1	68,153	68,152
CIFAR10 Superpixels	45,000	117.6	941.2	150	1,200
CiteSeer	3,327	13.0	42.3	64	488
COLLAB	5,000	74.5	4914.4	492	80,238
Cora	2,708	26.0	83.2	64	328
ENZYMES	600	32.6	124.3	126	298
FB15k-237	14,541	63.8	339.5	64	3,255
IEEE Power Grid	4,832	24.0	40.4	3,185	3,828
IMDB-BINARY	1,000	19.8	193.1	136	2,498
MNIST Superpixels	55,000	70.6	564.5	75	600
MUTAG	188	17.9	39.6	28	66
ogbg-code2	20,000	126.2	125.2	4,039	4,038
ogbg-molhiv	41,127	25.5	54.9	222	502
ogbg-molpcba	20,000	26.0	56.2	331	668
ogbg-ppa	200	270.5	5346.1	300	12,006
ogbn-arxiv	500	64.0	399.1	64	791
PROTEINS	1,113	39.1	145.6	620	2,098
PubMed	19,717	35.4	118.2	64	686
roadNet-CA	32,768	64.0	85.8	64	120
Synthetic Stress	42	10.7	15.3	16	28
WN18RR	40,943	64.0	90.9	64	189

5 Experiments

5.1 Experimental Setup

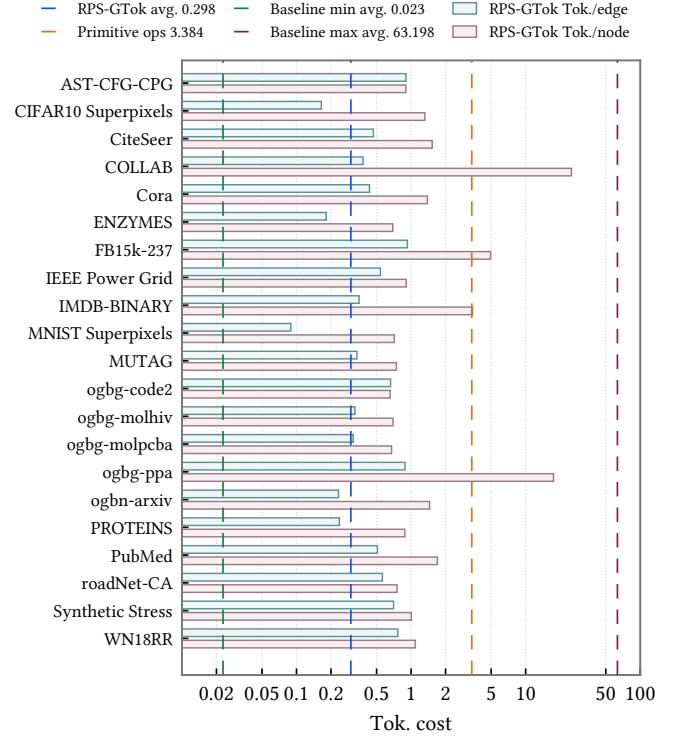
The token-level evaluation uses 21 graph datasets converted to attributed graphs [9, 20, 21, 28, 31, 35, 37, 40, 43]. Reconstructability applies Graph editing to each emitted graph-token sequence and compares the reconstructed graph with the source graph under S . Permutation stability is measured by relabeling incidental node identifiers and checking whether the emitted graph-token sequences remain identical.

The Transformer-based evaluation uses MUTAG, IMDB-BINARY, COLLAB, and PROTEINS with topology-derived targets rather than dataset labels: four classification targets over density bins, edge-count bins, node-count bins, and triangle existence, and four regression targets over density, edge count, node count, and triangle count. These downstream topological tasks probe node/edge coverage, global composition, and higher-order connectivity without conflating tokenizer quality with semantic supervision or dataset-specific statistical correlations. For regression, NMAE and NRMSE divide MAE and RMSE by the target standard deviation of each task, while NMSE divides MSE by the corresponding target variance before cross-task aggregation.

5.2 Token-Level Evaluation Results

We evaluate RPS-GTok on the 21 graph datasets summarized in Table 1. Figure 3 reports edge-normalized (Tok./edge) and node-normalized (Tok./node) costs. Tok./edge measures token cost per relation, whereas Tok./node also reflects graph density; dense datasets can therefore have low Tok./edge but high Tok./node.

Figure 3 shows that RPS-GTok keeps graph-token sequences compact across datasets, while the exact cost varies with graph density and repeated local structure. The average RPS-GTok costs across all 21 graph datasets are 0.298 Tok./edge and 1.522 Tok./node. These metrics measure the model-facing identifier sequence, while the lossless reconstruction footprint is determined by the token-local payloads and the expanded primitive-operation sequence. Compared with the primitive operation baseline at 3.384 Tok./edge,

**Figure 3: Per-dataset graph-token costs of RPS-GTok.****Table 2: Token-level comparison of graph tokenizers.**

Method	Tok./edge	Tok./node	Recon. (%)	Perm. (%)
RPS-GTok	0.660	2.829	100.00	100.00
edge list	1.237	5.299	100.00	0.00
adjacency list	2.036	8.724	100.00	0.00
graph6-like	1.362	5.837	19.79	80.20
text description	18.843	80.746	100.00	0.00
GraphML-like	63.198	270.810	100.00	0.00
tokenized JSON	23.890	102.372	100.00	0.00
spectral	0.023	0.098	0.00	100.00
WL subtree	0.213	0.913	0.00	100.00
frequent subgraph	0.029	0.122	0.00	100.00

RPS-GTok uses 8.8% of that identifier length while preserving the same Graph editing semantics.

Table 2 compares RPS-GTok with direct graph-representation baselines on 2,602 sampled attributed graphs. Recon. (%) measures reconstruction under S , and Perm. (%) measures stability under incidental node relabeling. RPS-GTok uses 0.660 Tok./edge and 2.829 Tok./node, with both audits equal to 100%. Among the direct representations, edge list, adjacency list, tokenized JSON, text description, and GraphML-like representations reconstruct the source graph, but they remain order-dependent. Graph6-like is only partially reconstructable, while spectral, WL subtree, and frequent-subgraph summaries are compact and permutation-stable but do not reconstruct the source graph.

These results show that RPS-GTok offers the best overall trade-off by achieving 100% reconstructability and 100% permutation stability at low token cost.

Table 3: Downstream topological classification tasks.

Method	Macro-F1	Accuracy	Precision	Recall	Specificity
edge-list BPE	0.9554 $\pm$ 0.0168	0.9593 $\pm$ 0.0140	0.9566 $\pm$ 0.0229	0.9564 $\pm$ 0.0103	0.9703 $\pm$ 0.0091
adjacency-list BPE	0.9513 $\pm$ 0.0035	0.9537 $\pm$ 0.0032	0.9558 $\pm$ 0.0028	0.9484 $\pm$ 0.0032	0.9688 $\pm$ 0.0016
DFS-order BPE	0.9542 $\pm$ 0.0062	0.9593 $\pm$ 0.0085	0.9634 $\pm$ 0.0099	0.9492 $\pm$ 0.0073	0.9631 $\pm$ 0.0108
BFS-order BPE	0.9572 $\pm$ 0.0137	0.9611 $\pm$ 0.0147	0.9636 $\pm$ 0.0140	0.9529 $\pm$ 0.0149	0.9668 $\pm$ 0.0093
Eulerian BPE	0.9664 $\pm$ 0.0034	0.9685 $\pm$ 0.0032	0.9719 $\pm$ 0.0027	0.9632 $\pm$ 0.0032	0.9762 $\pm$ 0.0016
GQT	0.9702 $\pm$ 0.0148	0.9722 $\pm$ 0.0147	0.9756 $\pm$ 0.0134	0.9669 $\pm$ 0.0147	0.9780 $\pm$ 0.0073
G2PT	0.9526 $\pm$ 0.0157	0.9556 $\pm$ 0.0147	0.9503 $\pm$ 0.0151	0.9562 $\pm$ 0.0155	0.9738 $\pm$ 0.0103
Graph-Tokenization-BPE	0.6681 $\pm$ 0.0188	0.6963 $\pm$ 0.0140	0.6800 $\pm$ 0.0321	0.7074 $\pm$ 0.0292	0.7916 $\pm$ 0.0234
RPS-GTok (Token-only)	0.9681 $\pm$ 0.0118	0.9704 $\pm$ 0.0116	0.9742 $\pm$ 0.0104	0.9651 $\pm$ 0.0116	0.9771 $\pm$ 0.0058
RPS-GTok (Full-Embed)	0.9796 $\pm$ 0.0065	0.9815 $\pm$ 0.0064	0.9839 $\pm$ 0.0061	0.9762 $\pm$ 0.0064	0.9826 $\pm$ 0.0032

Table 4: Downstream topological regression tasks.

Method	R^2	NMAE	NMSE	NRMSE	Spearman ρ
edge-list BPE	0.9891 $\pm$ 0.0038	0.0399 $\pm$ 0.0037	0.0084 $\pm$ 0.0029	0.0906 $\pm$ 0.0167	0.8929 $\pm$ 0.0078
adjacency-list BPE	0.9976 $\pm$ 0.0007	0.0237 $\pm$ 0.0026	0.0019 $\pm$ 0.0006	0.0430 $\pm$ 0.0070	0.9102 $\pm$ 0.0146
DFS-order BPE	0.9918 $\pm$ 0.0014	0.0312 $\pm$ 0.0009	0.0063 $\pm$ 0.0011	0.0792 $\pm$ 0.0069	0.8854 $\pm$ 0.0383
BFS-order BPE	0.9899 $\pm$ 0.0029	0.0383 $\pm$ 0.0061	0.0078 $\pm$ 0.0023	0.0872 $\pm$ 0.0130	0.8678 $\pm$ 0.0292
Eulerian BPE	0.9913 $\pm$ 0.0003	0.0332 $\pm$ 0.0035	0.0067 $\pm$ 0.0002	0.0819 $\pm$ 0.0015	0.9258 $\pm$ 0.0117
GQT	0.5459 $\pm$ 0.0156	0.2883 $\pm$ 0.0046	0.3512 $\pm$ 0.0121	0.5899 $\pm$ 0.0110	0.8419 $\pm$ 0.0423
G2PT	0.9542 $\pm$ 0.0037	0.0724 $\pm$ 0.0064	0.0354 $\pm$ 0.0029	0.1871 $\pm$ 0.0085	0.9074 $\pm$ 0.0495
Graph-Tokenization-BPE	0.6330 $\pm$ 0.0093	0.2141 $\pm$ 0.0017	0.2837 $\pm$ 0.0073	0.5278 $\pm$ 0.0090	0.9011 $\pm$ 0.0285
RPS-GTok (Token-only)	0.9984 $\pm$ 0.0003	0.0212 $\pm$ 0.0013	0.0012 $\pm$ 0.0002	0.0350 $\pm$ 0.0028	0.9006 $\pm$ 0.0317
RPS-GTok (Full-Embed)	0.9965 $\pm$ 0.0006	0.0279 $\pm$ 0.0043	0.0027 $\pm$ 0.0005	0.0521 $\pm$ 0.0044	0.9411 $\pm$ 0.0138

5.3 Transformer-Based Evaluation on Downstream Topological Tasks

All Transformer-based evaluations use a fixed encoder-only Transformer architecture and the same training protocol across token views. For the compared baselines and RPS-GTok (Token-only), the Transformer architecture, token embeddings, position embeddings, splits, training budgets, and task definitions are fixed; the varying component is the graph-to-token conversion. The shared Transformer setting uses a 512-token context budget, 6 encoder layers, hidden size 256, 8 attention heads, feed-forward width 1024, GELU activation, dropout 0.10, and the same truncation rule across token views. For RPS-GTok (Full-Embed), only the token adapter before the Transformer changes.

Tables 3 and 4 organize the compared methods into three groups: structure serializations with BPE, external tokenizer baselines, and RPS-GTok variants. These methods are reported separately because they use different tokenizer implementations, vocabulary construction procedures, and truncation behavior under the same 512-token budget. The Transformer-based evaluation tests whether a fixed Transformer can access graph-topological information from the emitted token identifiers on downstream topological tasks.

The BPE serializations are generally compact and competitive, but they inherit reconstructability and permutation-stability limitations from their serialization orders. Among external baselines, GQT is strong on classification but weaker on count-sensitive regression, whereas the long prompt-style sequences of G2PT and

Eulerian-style sequences of Graph-Tokenization-BPE limit their effectiveness under the 512-token budget.

In contrast, Tables 3 and 4 show that RPS-GTok graph tokens are effective even without additional adaptation components. RPS-GTok (Token-only) nearly achieves the best regression results and reaches the top three on classification tasks. Adding the token adapter in RPS-GTok (Full-Embed) further achieves the best classification performance.

These results indicate that RPS-GTok exposes graph-topological information more effectively than competing tokenization methods on downstream topological tasks.

6 Conclusion

In this paper, we introduce RPS-GTok, a graph tokenization framework that bridges attributed graphs with Transformer-based models. RPS-GTok constructs discrete graph tokens whose primitive-operation expansions support reconstructability and whose emitted sequence satisfies permutation stability. Fundamentally, RPS-GTok decouples graph token generation from the downstream model architecture and provides Transformer-based models with a stable, reconstructable sequence interface to graph topology. Empirically, on 21 attributed graph datasets and 2,602 sampled graphs, RPS-GTok achieves compact tokenization while preserving reconstructability and permutation stability, and RPS-GTok improves Transformer-based model performance on downstream topological classification and regression tasks.

References

- [1] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. 2018. Relational Inductive Biases, Deep Learning, and Graph Networks. *arXiv preprint arXiv:1806.01261* abs/1806.01261 (2018), 38 pages. <https://arxiv.org/abs/1806.01261>
- [2] Ulrik Brandes, Markus Eiglsperger, Ivan Herman, Michael Himsolt, and M. Scott Marshall. 2002. GraphML Progress Report: Structural Layer Proposal. In *Graph Drawing (Lecture Notes in Computer Science, Vol. 2265)*. Springer, Berlin, Heidelberg, 501–512. doi:10.1007/3-540-45848-4_59
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Virtual Event, 1877–1901.
- [4] Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. 2023. NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs. In *International Conference on Learning Representations*. OpenReview.net, Kigali, Rwanda, 20 pages. <https://openreview.net/forum?id=8KYeIT3Ow>
- [5] Xiaohui Chen, Yinkai Wang, Jiaxing He, Yuanqi Du, Soha Hassoun, Xiaolin Xu, and Li-Ping Liu. 2025. Graph Generative Pre-trained Transformer. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Vancouver, BC, Canada, 9176–9197. <https://proceedings.mlr.press/v267/chen25bl.html>
- [6] Yongqiang Chen, Quanming Yao, Juzheng Zhang, James Cheng, and Yatao Bian. 2025. Hierarchical Graph Tokenization for Molecule-Language Alignment. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Vancouver, Canada, 9664–9690. <https://proceedings.mlr.press/v267/chen25cf.html>
- [7] Richard Cyganiak, David Wood, and Markus Lanthaler. 2014. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation. World Wide Web Consortium. <https://www.w3.org/TR/rdf11-concepts/>
- [8] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *Advances in Neural Information Processing Systems*, Vol. 29. Curran Associates, Inc., Barcelona, Spain, 3844–3852.
- [9] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *AAAI Conference on Artificial Intelligence*. AAAI Press, New Orleans, LA, USA, 1811–1818.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Conference of the North American Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, Minneapolis, MN, USA, 4171–4186. doi:10.18653/v1/N19-1423
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*. OpenReview.net, Virtual Event, 22 pages. <https://openreview.net/forum?id=YicbFdNTTy>
- [12] David K. Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alan Aspuru-Guzik, and Ryan P. Adams. 2015. Convolutional Networks on Graphs for Learning Molecular Fingerprints. In *Advances in Neural Information Processing Systems*, Vol. 28. Curran Associates, Inc., Montreal, QC, Canada, 2224–2232.
- [13] Vijay Prakash Dwivedi and Xavier Bresson. 2020. A Generalization of Transformer Networks to Graphs. *arXiv preprint arXiv:2012.09699* abs/2012.09699 (2020), 12 pages. <https://arxiv.org/abs/2012.09699>
- [14] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk Like a Graph: Encoding Graphs for Large Language Models. In *International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 20 pages. <https://openreview.net/forum?id=IuXR1CCrSi>
- [15] Philip Gage. 1994. A New Algorithm for Data Compression. *The C Users Journal* 12, 2 (1994), 23–38. <https://dl.acm.org/doi/10.5555/177910.177914>
- [16] Emden R. Gansner and Stephen C. North. 2000. An Open Graph Visualization System and Its Applications to Software Engineering. *Software: Practice and Experience* 30, 11 (2000), 1203–1233.
- [17] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, Sydney, NSW, Australia, 1263–1272.
- [18] Zeyuan Guo, Enmao Diao, Cheng Yang, and Chuan Shi. 2026. Graph Tokenization for Bridging Graphs and Transformers. In *International Conference on Learning Representations*. OpenReview.net, Rio de Janeiro, Brazil, 21 pages. <https://openreview.net/forum?id=jCtXt1BGF>
- [19] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., Long Beach, CA, USA, 1024–1034.
- [20] Weihua Hu, Matthias Fey, Hongyu Ren, Maho Nakata, Yuxiao Dong, and Jure Leskovec. 2021. OGB-LSC: A Large-Scale Challenge for Machine Learning on Graphs. In *NeurIPS Datasets and Benchmarks*. Curran Associates, Inc., Virtual Event, 8 pages.
- [21] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Virtual Event, 22118–22133.
- [22] Bowen Jin, Chulin Xie, Jiawei Zhang, Kashob Kumar Roy, Yu Zhang, Zheng Li, Ruirui Li, Xianfeng Tang, Suhang Wang, Yu Meng, and Jiawei Han. 2024. Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 163–184. doi:10.18653/v1/2024.findings-acl.11
- [23] Jinwoo Kim, Tien Dat Nguyen, Seonwoo Min, Sungjun Cho, Moontae Lee, Honglak Lee, and Seunghoon Hong. 2022. Pure Transformers are Powerful Graph Learners. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., New Orleans, LA, USA, 14582–14595.
- [24] Wooyoung Kim, Byungyoon Park, and Wooju Kim. 2025. Query-Aware Learnable Graph Pooling Tokens as Prompt for Large Language Models. *arXiv preprint arXiv:2501.17549* abs/2501.17549 (2025), 17 pages. <https://arxiv.org/abs/2501.17549>
- [25] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*. OpenReview.net, Toulon, France, 14 pages. <https://openreview.net/forum?id=SJU4ayYgl>
- [26] Mario Krenn, Florian Häse, AkshatKumar Nigam, Pascal Friederich, and Alán Aspuru-Guzik. 2020. SELFIES: A Robust Representation of Semantically Constrained Graphs with an Example Application in Chemistry. *Machine Learning: Science and Technology* 1, 4 (2020), 045024. doi:10.1088/2632-2153/aba947
- [27] Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. 2021. Rethinking Graph Transformers with Spectral Attention. In *Advances in Neural Information Processing Systems*, Vol. 34. Curran Associates, Inc., Virtual Event, 21618–21629.
- [28] Alex Krizhevsky. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.
- [29] Taku Kudo and John Richardson. 2018. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Brussels, Belgium, 66–71. doi:10.18653/v1/D18-2012
- [30] Michihiro Kuramochi and George Karypis. 2001. Frequent Subgraph Discovery. In *IEEE International Conference on Data Mining*. IEEE Computer Society, San Jose, CA, USA, 313–320.
- [31] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. doi:10.1109/5.726791
- [32] Gang Liu, Jie Chen, Yihan Zhu, Michael Sun, Tengfei Luo, Nitesh V. Chawla, and Meng Jiang. 2026. Graph Diffusion Transformers are In-Context Molecular Designers. In *International Conference on Learning Representations*. OpenReview.net, Rio de Janeiro, Brazil, 24 pages. <https://openreview.net/forum?id=IJ87GN5zJc>
- [33] Brendan D. McKay and Adolfo Piperno. 2014. Practical Graph Isomorphism, II. *Journal of Symbolic Computation* 60 (2014), 94–112. doi:10.1016/j.jsc.2013.09.003
- [34] Grégoire Mialon, Dexiong Chen, Margot Selosse, and Julien Mairal. 2021. GraphiT: Encoding Graph Structure in Transformers. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, Virtual Event, 7470–7479.
- [35] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A Collection of Benchmark Datasets for Learning with Graphs. *arXiv preprint arXiv:2007.08663* abs/2007.08663 (2020), 14 pages. <https://arxiv.org/abs/2007.08663>
- [36] Ladislav Rampasek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a General, Powerful, Scalable Graph Transformer. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., New Orleans, LA, USA, 14501–14515.
- [37] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI Conference on Artificial Intelligence*. AAAI Press, Austin, TX, USA, 4292–4293.
- [38] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Transactions on Neural Networks* 20, 1 (2009), 61–80. doi:10.1109/TNN.2008.2005605
- [39] Mike Schuster and Kaisuke Nakajima. 2012. Japanese and Korean Voice Search. In *IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, Kyoto, Japan, 5149–5152. doi:10.1109/ICASSP.2012.6289079
- [40] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–106. doi:10.1609/aimag.v29i3.2157

- [41] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Berlin, Germany, 1715–1725. doi:10.18653/v1/P16-1162
- [42] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. 2011. Weisfeiler-Lehman Graph Kernels. *Journal of Machine Learning Research* 12 (2011), 2539–2561.
- [43] Kristina Toutanova and Danqi Chen. 2015. Observed versus Latent Features for Knowledge Base and Text Inference. In *Workshop on Continuous Vector Space Models and their Compositionality*. Association for Computational Linguistics, Beijing, China, 57–66. doi:10.18653/v1/W15-4007
- [44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., Long Beach, CA, USA, 5998–6008.
- [45] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*. OpenReview.net, Vancouver, BC, Canada, 12 pages. <https://openreview.net/forum?id=rjXmpikCZ>
- [46] Limei Wang, Kaveh Hassani, Si Zhang, Dongqi Fu, Baichuan Yuan, Weilin Cong, Zhigang Hua, Hao Wu, Ning Yao, and Bo Long. 2025. Learning Graph Quantized Tokenizers. In *International Conference on Learning Representations*. OpenReview.net, Singapore, 33 pages. <https://openreview.net/forum?id=oYSsbY3G4o>
- [47] David Weininger. 1988. SMILES, a Chemical Language and Information System. 1. Introduction to Methodology and Encoding Rules. *Journal of Chemical Information and Computer Sciences* 28, 1 (1988), 31–36. doi:10.1021/ci00057a005
- [48] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*. OpenReview.net, New Orleans, LA, USA, 22 pages. <https://openreview.net/forum?id=ryGs6iA5Km>
- [49] Xifeng Yan and Jiawei Han. 2002. gSpan: Graph-Based Substructure Pattern Mining. In *IEEE International Conference on Data Mining*. IEEE Computer Society, Maebashi City, Japan, 721–724.
- [50] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do Transformers Really Perform Badly for Graph Representation?. In *Advances in Neural Information Processing Systems*, Vol. 34. Curran Associates, Inc., Virtual Event, 28877–28888.
- [51] Jianan Zhao, Le Zhuo, Yikang Shen, Meng Qu, Kai Liu, Michael Bronstein, Zhaocheng Zhu, and Jian Tang. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089* abs/2310.01089 (2023), 17 pages. <https://arxiv.org/abs/2310.01089>

A Graph Canonical Relabeling Implementation

The implementation of graph canonical relabeling follows an attributed canonical-labeling procedure. For each node tuple v , RPS-GTok first forms the node-content signature $b_v = (\tau_v, Q(A_v), R(A_v))$. Here τ_v denotes the node type, A_v denotes the retained attribute multiset of v , $Q(\cdot)$ is the deterministic attribute-quantization map, and $R(\cdot)$ is the deterministic retained-attribute serializer. Attribute fields are serialized into deterministic key-value strings under the schema-fixed formatting rules, while the quantized view is generated separately by $Q(\cdot)$. Therefore, the initial node ordering key does not depend on the input row position of v or on its raw node identifier.

For each node pair (u, v) , RPS-GTok collects all edge tuples between the two node tuples. Each collected edge tuple is represented by an edge descriptor $(\eta, \tau_e, \delta_e, k_e, Q(A_e), R(A_e))$, where η records whether the edge is viewed as outgoing, incoming, or undirected from the current endpoint. Here τ_e denotes the edge type, δ_e denotes the direction flag, k_e denotes a stable multiedge key, $Q(A_e)$ denotes the quantized edge-attribute representation, and $R(A_e)$ denotes the deterministic retained-attribute serialization for the edge. If the input graph does not provide a stable parallel-edge identity, k_e is generated as a deterministic rank after sorting parallel-edge descriptors by retained edge content. The descriptors between the same node pair are ordered by lexicographic tuple comparison. This sorted descriptor list is the pair signature p_{uv} . Parallel edges with the same endpoints are therefore compared by endpoint roles, edge types, direction flags, stable multiedge keys, quantized edge attributes, and raw retained edge-attribute serialization, rather than by input edge order.

RPS-GTok then refines node ordering keys by repeatedly combining the current key of each node tuple with the sorted neighboring keys and pair signatures. This is the implementation counterpart of the KeyUpdate step. The refinement separates node tuples whenever their attributed local structure becomes distinguishable. When an update no longer changes the ordering keys, the keys are treated as stable.

If stable keys still leave tied node tuples, RPS-GTok applies an individualization-refinement search inside the tied groups. The implementation temporarily places one tied node tuple into a singleton position, refines the keys again, and recursively compares the candidate orders produced by these choices. For each candidate order, RPS-GTok writes a projected adjacency string: node tuples are written in candidate-rank order, each edge endpoint is represented by the rank of its endpoint node tuple, and retained edge attributes are serialized through both $Q(A_e)$ and $R(A_e)$. The candidate with the lexicographically smallest projected adjacency string is selected as the canonical node order. Canonical node identifiers n_i are then assigned according to this order, and edge tuples are rewritten with these canonical identifiers.

A.1 Candidate Scoring and Retention Details

This subsection gives the implementation-level scoring rule used by Algorithm 1 to instantiate Eq. (25). For a candidate U , let $N_E(U) = E_{\text{in}}(U) \setminus C_E$ be the newly covered internal edge tuples. The retained

implementation score is

$$\Delta_{\text{impl}}(U \mid C_E, \mathcal{U}_{\text{ret}}) = \lambda_E |N_E(U)| + \tilde{s}_{\text{loc}}(U) + \lambda_H h_\theta(U) + b_{\text{sem}}(U, C_E) - \lambda_J J(U, \mathcal{U}_{\text{ret}}). \quad (36)$$

Here $h_\theta(U) \in \{0, 1\}$ indicates whether the candidate matches a graph-vocabulary entry in $\mathcal{A}_\theta$. The weights used in the reported experiments are $\lambda_E = 2.5$, $\lambda_H = 0.4$, and $\lambda_J = 0.6$, corresponding to the fixed coverage, graph-vocabulary, and overlap weights in the tokenizer configuration.

The static local score is

$$\tilde{s}_{\text{loc}}(U) = 0.3 d(U) + 0.3 c_\tau(U) + 0.2 c_A(U) - 0.5 p(U) - 0.5 r_{\text{ood}}(U) + b_\ell(U). \quad (37)$$

In Eq. (37), $d(U) = |E_{\text{in}}(U)| / (|U| \max(1, |U| - 1))$ is the directed-density score used by the candidate generator, $c_\tau(U)$ is the fraction of nodes in U that share the most frequent node type, and $c_A(U)$ is the Jaccard consistency of selected quantized node-attribute token sets. If no selected attribute tokens are available, $c_A(U) = 0$. The port penalty is $p(U) = |P(U)| / \max(1, b_0)$, where $b_0 = 8$ is the base boundary-port budget. The OOD term $r_{\text{ood}}(U)$ is the fraction of selected quantized node-attribute entries whose quantizer fallback flag marks them as out-of-distribution. The source-category bonus $b_\ell(U)$ is 0.35 for semantic candidate sources, 0.05 for edge-fallback or isolated-node candidates, and 0 otherwise.

The semantic source bonus in Eq. (36) is

$$b_{\text{sem}}(U, C_E) = \begin{cases} 0.25 |N_E(U)| + 0.05 |U|, & \ell(U) \in \mathcal{L}_{\text{sem}}, |U| \geq 3, \\ 0, & \text{otherwise,} \end{cases} \quad (38)$$

where $\mathcal{L}_{\text{sem}}$ contains hub-star, typed-star, dense-core, bridge-block, and multiedge-bundle candidates. The overlap penalty is the maximum node-set Jaccard overlap with already retained candidates,

$$J(U, \mathcal{U}_{\text{ret}}) = \max_{U' \in \mathcal{U}_{\text{ret}}} \frac{|U \cap U'|}{\max(1, |U \cup U'|)}, \quad (39)$$

with value 0 when $\mathcal{U}_{\text{ret}}$ is empty. If a non-fallback candidate has $J > 0.7$, it is skipped before scoring. The default candidate generation limits are $m = 6$, $b_0 = 8$, target edge coverage 0.98, and a candidate budget equal to $|V| + |E|$ when no smaller graph-level budget is specified. Domain-specific configurations may raise m or the source port limit for high-degree graph families, but the score definitions in Eqs. (36)–(39) remain fixed.

B Computational Considerations for Canonical Relabeling

The main text gives the computational boundary of primitive compilation and fragment formation, and states that graph canonical relabeling may branch on highly symmetric graphs. This appendix clarifies that point as a computational limitation rather than an additional experimental protocol. The canonical relabeling procedure is deterministic, but determinism does not imply a uniform polynomial-time bound for every attributed graph admitted by $\mathcal{S}$. Refinement typically separates many node tuples through their types, retained attributes, edge descriptors, and neighborhood signatures. When these signals leave large tied groups, the individualization-refinement search may need to compare multiple candidate orders before selecting the lexicographically minimal canonical serialization.

Table 5: Component ablation of RPS-GTok compactness and reconstructability.

Setting	Tok./edge	Tok./node	Recon.	Prim. (%)
Full RPS-GTok	0.660	2.829	1.000	0.03
primitive operations	3.384	17.286	1.000	100.00
no graph vocabulary	0.849	3.636	1.000	0.96
no fragment	1.267	5.430	1.000	0.04
primitive fragment only	0.660	2.829	0.923	0.00

This limitation affects runtime rather than the definition of the emitted graph tokens. Once canonical relabeling completes, the subsequent primitive compilation, vocabulary-guided primitive fragment formation, and graph-token emission remain deterministic and preserve the reconstructability and permutation-stability properties stated in the main text. In practice, the most demanding cases are graphs with large automorphism groups or sparse retained attributes, where many nodes are structurally indistinguishable under the prescribed graph paradigm. The framework therefore makes a bounded claim: the primitive and fragment stages have the explicit costs reported in Eq. (35), while the canonical relabeling stage is input-sensitive and can dominate the total tokenization time on highly symmetric inputs.

C Additional Experimental Results

Table 5 separates compactness from reconstruction mechanisms. Tok./edge and Tok./node denote emitted graph-token counts normalized by edge tuples and node tuples, respectively; they measure the model-facing identifier footprint. Recon. denotes the reconstruction score under S ; and Prim. (%) is the percentage of emitted entries represented as primitive-operation entries rather than vocabulary-matched fragment tokens, so it serves as the direct lossless-footprint indicator in this table. Primitive-operation encodings remain reconstructable but long, with 3.384 Tok./edge, 17.286 Tok./node, and 100.00% primitive tokens. Full RPS-GTok reduces primitive-operation entries to 0.03%. Removing the graph vocabulary or fragment formation increases token cost, while primitive-fragment-only encoding reduces reconstructability to 0.923. This shows that compactness comes from vocabulary-matched primitive fragments, and uncovered graph information still requires primitive operations.

The parameter sensitivity study varies one tokenizer parameter group at a time while keeping the prescribed graph paradigm S , graph canonical relabeling, primitive operations, graph vocabulary construction procedure, and reconstruction audit fixed. Table 6 reports the resulting token cost. The Axis column identifies the varied parameter group, and the Compared values column gives the sweep range. Tok./edge and Tok./node are the mean numbers of emitted graph tokens normalized by edge tuples and node tuples, respectively. P95 L is the 95th percentile of the emitted graph-token sequence length L , where $Z = (z_1, \dots, z_L)$. Changes in Table 6 therefore measure compactness rather than reconstructability or permutation stability. Fragment formation has the largest effect: without fragment-based merging, Tok./edge rises from 0.652 to 1.245 and P95 L rises from 75 to 114. Larger substructures and more boundary endpoints give smaller gains, while vocabulary size

Algorithm 2: Transformer Inputs from RPS-GTok Graph Tokens

Input: Emitted graph-token sequence $Z = (z_1, \dots, z_L)$, where $z_i = (x_i, \alpha_i)$
Output: Token-only inputs $\{h_i^{\text{tok}}\}_{i=1}^L$ and Full-Embed inputs $\{h_i^{\text{full}}\}_{i=1}^L$

```

1 for  $i = 1, \dots, L$  do
2   Read the model-facing vocabulary identifier  $x_i$  from  $z_i$ 
3   Set the token position to  $i$ 
4   Compute  $h_i^{\text{tok}} \leftarrow E_x(x_i) + E_{\text{pos}}(i)$ 
5   Extract  $\beta_i$  from  $\alpha_i$  and  $\exp_{\theta}(z_i)$ 
6   Compute  $h_i^{\text{full}} \leftarrow E_x(x_i) + E_{\text{pos}}(i) + E_{\beta}(\beta_i)$ 
7 return  $\{h_i^{\text{tok}}\}_{i=1}^L$  and  $\{h_i^{\text{full}}\}_{i=1}^L$ 

```

and Transformer window length do not change tokenizer compactness in this sample. This pattern suggests that compression mainly comes from aligning recurring primitive fragments with vocabulary entries, rather than from increasing vocabulary capacity or Transformer context length.

Table 7 reports auxiliary mean-length and memory statistics recorded in the same Transformer-based evaluation as Tables 3 and 4, using the same method ordering. Eulerian BPE is the local standardized Eulerian serialization with BPE, whereas Graph-Tokenization-BPE is evaluated as an external tokenizer baseline through its official token generator with local input/output adapters. GQT has the shortest sequences and lowest memory footprint because it emits compact vector-quantized node codes, while edge-list BPE is the lightest direct serialization baseline. RPS-GTok uses longer inputs than these compact baselines because its emitted graph tokens preserve reconstructable topology, but it remains substantially shorter and less memory-intensive than prompt-style or long textual tokenization baselines such as G2PT and Graph-Tokenization-BPE.

D Transformer-Based Model Details

The Transformer input layer consumes the identifier projection of the emitted graph-token sequence $Z = (z_1, \dots, z_L)$ after the tokenizer-level audit. For each graph token $z_i = (x_i, \alpha_i)$, x_i is the model-facing vocabulary identifier and α_i stores the minimal token-local arguments used by the frozen expansion map. The Token-only setting uses only token identifiers and sequence positions:

$$h_i^{\text{tok}} = E_x(x_i) + E_{\text{pos}}(i), \quad (40)$$

where h_i^{tok} is the input embedding of the i -th graph token, E_x is the token-identifier embedding table, and E_{pos} is the position embedding table.

The Full-Embed setting uses the same token identifiers and adds side information derived from α_i and $\exp_{\theta}(z_i)$. Let β_i denote these derived fields, including token role, fragment-local position, expansion length, port count, retained-attribute indicators, and numeric attribute payload features when present. These fields are derived from the emitted graph token and its primitive-operation expansion, and therefore do not introduce graph information outside the prescribed graph paradigm S . The Full-Embed input is

$$h_i^{\text{full}} = E_x(x_i) + E_{\text{pos}}(i) + E_{\beta}(\beta_i), \quad (41)$$

where h_i^{full} is the input embedding of the i -th graph token in the Full-Embed setting, and E_{β} maps the derived side information to the Transformer hidden space.

Table 6: Parameter sensitivity of RPS-GTok graph tokenization.

Axis	Compared values	Tok./edge	Tok./node	P95 L
graph vocabulary size	1K $\rightarrow$ 64K	0.652 $\rightarrow$ 0.652	2.693 $\rightarrow$ 2.693	75 $\rightarrow$ 75
fragment rules	0 $\rightarrow$ 1K	1.245 $\rightarrow$ 0.652	5.146 $\rightarrow$ 2.693	114 $\rightarrow$ 75
max fragment size	2 $\rightarrow$ 16	0.665 $\rightarrow$ 0.641	2.749 $\rightarrow$ 2.647	83 $\rightarrow$ 75
max boundary endpoints	2 $\rightarrow$ 64	0.662 $\rightarrow$ 0.632	2.736 $\rightarrow$ 2.611	75 $\rightarrow$ 72
max sequence length	128 $\rightarrow$ 2048	0.652 $\rightarrow$ 0.652	2.693 $\rightarrow$ 2.693	75 $\rightarrow$ 75

Table 7: Auxiliary mean-length and memory statistics for downstream topological tasks.

Method	Cls. Mean len.	Cls. Memory (MB)	Reg. Mean len.	Reg. Memory (MB)
edge-list BPE	<u>35.9833</u>	<u>619.4808</u>	<u>32.5556</u>	<u>591.7930</u>
adjacency-list BPE	44.6000	939.7653	43.8222	942.2155
DFS-order BPE	89.9944	1869.4466	82.0667	1955.0990
BFS-order BPE	76.6444	1894.7843	69.7667	2012.2765
Eulerian BPE	148.4444	2392.4536	119.7889	2392.4648
GQT	<u>21.6611</u>	<u>267.9116</u>	<u>20.7667</u>	<u>267.6963</u>
G2PT	502.7167	2456.9702	406.6556	2456.9639
Graph-Tokenization-BPE	446.0111	2456.8198	352.3556	2456.8115
RPS-GTok (Token-only)	87.2167	1817.8447	67.9333	1817.8208
RPS-GTok (Full-Embed)	87.2167	1707.3590	67.9333	1986.3018

Formally, the token-local side-information extractor is

$$\beta_i = f_\beta(\alpha_i, \exp_\theta(z_i)), \quad (42)$$

where f_β is deterministic and uses only information contained in the emitted graph token z_i and its primitive-operation expansion. Equation (42) makes explicit that Full-Embed augments Token-only with token-local derived fields; it does not add graph information outside the prescribed graph paradigm S . Here, graph editing refers to the deterministic state update that reconstructs the graph from primitive operations.

Algorithm 2 maps the emitted graph-token sequence to Transformer inputs in the two downstream topological evaluation settings. Its input is the complete graph-token sequence $Z = (z_1, \dots, z_L)$, where each token $z_i = (x_i, \alpha_i)$ carries a model-facing identifier x_i and a token-local payload α_i . The Token-only setting uses x_i and the position i , so it measures what topology is recoverable from the identifier sequence alone. The Full-Embed setting keeps the same token order and identifiers but augments them with β_i , a deterministic collection of token-local fields derived from α_i and $\exp_\theta(z_i)$. Thus, the two settings differ only in whether the input layer observes token-derived payload features beyond the identifier sequence.

Table 8 summarizes the encoder-only Transformer configuration used in the downstream topological tasks. Hidden size specifies the model width, attention heads specifies the number of multi-head self-attention heads, and feed-forward width specifies the inner dimension of the Transformer feed-forward block. The pooling setting concatenates the first-token, mean, and max pooled representations before the prediction head, and Linear(768, 256) denotes the first projection from the 768-dimensional pooled representation to 256 hidden units.

Table 8: Encoder-only Transformer parameters for downstream topological tasks.

Parameter	Value
Architecture	encoder-only Transformer
Context budget	512 tokens
Layers	6
Hidden size	256
Attention heads	8
Feed-forward width	1024
Activation	GELU
Dropout	0.10
Normalization	pre-normalization
Input layout	batch-first
Pooling	first-token, mean, and max pooling concatenated
Prediction head	Linear(768, 256), GELU, dropout, output layer

E Distinctions from External Graph Tokenizers

This section clarifies the tokenizer-level distinctions between RPS-GTok and the external graph tokenizers used in the comparison. These methods share the broad goal of making graphs consumable by Transformer-based models, but they define tokens over different objects and therefore assign different meanings to discreteness, reconstruction, and stability.

GQT [46] discretizes node representations encoded by a GNN through residual vector quantization and emits codebook indices for latent node states. Its tokens are compact discrete codes, but the code semantics are inherited from neural representation learning. A GQT identifier therefore denotes a latent embedding region rather than an executable graph-editing fragment, and its token sequence is not designed as a primitive-operation program whose expansion is audited against the retained topology under S .

G2PT [5] follows a prompt-style graph-text design. It exposes graph information to Transformer-based models through textualized traversal or graph descriptions. This makes the representation closer to graph serialization than to graph-token construction: the sequence is formed from syntax tokens, node-index mentions, traversal text, and prompt scaffolding. The resulting token stream can carry graph information, but its token semantics are textual rather than primitive-operational, and the sequence length is strongly affected by serialization syntax.

Graph-Tokenization-BPE [18] is the closest external baseline because it also combines graph serialization with BPE-style sub-word formation. However, its reversible and deterministic claims are attached to graph serialization rather than to graph-token instances with executable graph-editing semantics. The core object is a serialization string: reversibility is defined for the serialized graph description, and BPE merges frequent substrings in that string. This leaves two limitations relative to the tokenizer-level contract used here. First, the BPE unit is a text fragment induced by frequency in a serialized stream, so it does not carry local graph-editing pre-conditions or a primitive-operation expansion by construction. Second, deterministic traversal, start-node selection, tie-breaking, and equivalence up to cyclic shifts are serialization-level notions; they do not by themselves establish equality of the complete ordered graph-token sequence emitted by $T_{S, \mathcal{A}_\theta}$ under Eq. (34).

RPS-GTok takes a different design route. It first fixes the retained graph information through the prescribed graph paradigm S , then uses graph canonical relabeling to remove dependence on incidental node identifiers before token emission. Its graph token is a vocabulary-indexed primitive-fragment instance $z = (x, \alpha)$: the identifier x is the model-facing symbol, and the payload α stores the minimal token-local arguments required by the frozen expansion map. In contrast to GQT, x is not a latent node-state code; its meaning is tied to an expansion into primitive operations. In contrast to G2PT, the sequence is not a prompt or textual graph description; syntax and node-index text are not the primary carrier of graph structure. The central distinction from Graph-Tokenization-BPE is more fundamental than the choice of merge algorithm. Graph-Tokenization-BPE applies BPE after a reversible serialization has already linearized the graph, so the learned unit remains a substring of that linear text. RPS-GTok instead compiles the canonical graph into a reconstructable primitive-operation program before fragment formation. A vocabulary entry is therefore a reusable primitive fragment, and a complete token instance expands through $\text{Exp}_\theta(Z)$ into operations that are executed and checked by Graph editing under S . This makes Graph editing and Reconstructable Primitive Compilation part of the token semantics, rather than a post hoc interpretation of a serialized string.

The comparison therefore separates latent code discretization, prompt-style graph text, reversible serialization with BPE, and reconstructable primitive-fragment tokenization. RPS-GTok is designed for the last setting. Its distinctive point is the combination of a prescribed reconstruction contract, canonical token emission, vocabulary-guided primitive fragments, and token-local expansion arguments. This design makes the graph-token sequence consumed by the Transformer-based model independent of incidental node identifiers while keeping the complete graph-token instances tied to an explicit reconstruction path.

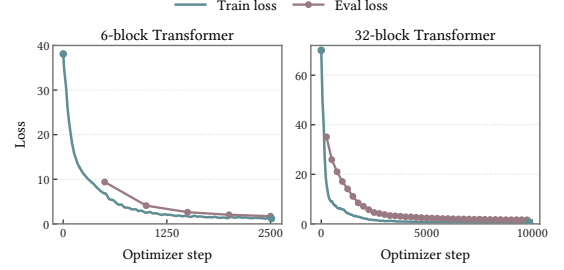


Figure 4: Autoregressive Transformer learning on graph token sequences under the next-token objective in Eq. (43).

F Autoregressive Transformer Learning Experiments

Section 5 evaluates downstream topological tasks. This appendix section instead reports an autoregressive training diagnostic on the emitted graph-token sequences. The objective is to predict the next graph-token identifier from an observed prefix, which tests whether RPS-GTok produces sequences with learnable autoregressive structure before any task-specific prediction head is introduced. Let p_ϕ be a Transformer parameterized by ϕ . For a graph-token sequence $Z = (z_1, \dots, z_L)$ with identifiers $x_1, \dots, x_L$, the next-token objective is

$$\mathcal{L}_{\text{seq}}(\phi) = -\frac{1}{L-1} \sum_{i=1}^{L-1} \log p_\phi(x_{i+1} | x_1, \dots, x_i). \quad (43)$$

In Eq. (43), $\mathcal{L}_{\text{seq}}$ is the autoregressive sequence loss, x_{i+1} is the next model-facing graph-token identifier, and $x_1, \dots, x_i$ is the observed prefix. The objective operates only on graph-token identifiers and does not alter graph canonical relabeling, primitive operations, graph vocabulary entries, or primitive-operation expansions.

Figure 4 plots training and evaluation loss against optimizer step. The 6-block Transformer reduces training loss from 38.10 to 1.12, with evaluation loss decreasing from 9.40 to 1.72. The 32-block Transformer shows the same trend at larger depth, reducing training loss from 70.03 to 0.58 and evaluation loss from 35.14 to 1.64. These curves show that RPS-GTok yields graph-token sequences with learnable autoregressive structure, while the tokenizer-level properties established earlier remain unchanged.